

**Department of Computer Engineering COMSATS University
Islamabad, Attock Campus**

Lab Rubrics Form	
Lab Name	Analysis of Discrete-Time System Properties – Memory, Causality, Linearity, and Time-Invariance
Lab Number	07
Submitted by:	
Student Names	Registration #
Sumaira Safeer	FA22-BCE-019

Rubrics		Marks		
		1	2	3
PLO-5 Modern Tools Usage	Ability to execute the experiment utilizing hardware, software tools, or a combination of both.			
PLO-10 Communication Skills	Present lab activities effectively through detailed lab reports, oral examinations, and engaging presentations.			

Lab-07

Analysis of Discrete-Time System Properties – Memory, Causality, Linearity, and Time-Invariance

Lab Task:

Find if Discrete Time System Described by the I/O relationship: $y[n] = -x[n]$

1. Part (a): Determine if the system is Static or Dynamic

Input signal: $x[n] = 2n$ for $-2 \leq n \leq 2$

2. Part (b): Determine if the system is Causal or Non-causal

3. Part (c): Determine if the system is Linear or Non-linear

Input signals:

i. $x_1[n] = 2n$ for $-2 \leq n \leq 2$

ii. $x_2[n] = n/3$ for $-2 \leq n \leq 2$, Constants: $a_1 = 2$, $a_2 = 3$

4. Part (d): Determine if the system is Shift Invariant or Shift Variant

Input signal: $x[n] = 2n$ for $-2 \leq n \leq 2$, Shift: $n_0 = 3$

Code:

```
clc; clear; close all;
%INPUT SIGNAL
n = -2:2;
x = 2*n;      % x[n] = 2n
y = -x;       % System y[n] = -x[n]
disp('Input x[n]:');
disp([n' x'])
disp('Output y[n] = -x[n]:');
disp([n' y'])
%PART (a) STATIC / DYNAMIC
fprintf('\n(a) Checking Static or Dynamic...\n');
fprintf('System is STATIC (memoryless) because y[n] depends only on x[n].\n');
figure;
stem(n, x, 'filled', 'LineWidth', 2); hold on;
stem(n, y, 'filled', 'LineWidth', 2);
title('Static / Dynamic Test: Input vs Output');
xlabel('n'); ylabel('Amplitude');
legend('Input x[n]', 'Output y[n] = -x[n]');
```

```

grid on;
xticks(-2:2);
%PART (b) CAUSAL / NON-CAUSAL
fprintf('\n(b) Checking Causality...\n');
fprintf('System is CAUSAL because output uses only present input x[n].\n');
figure;
stem(n, x, 'filled', 'LineWidth', 2); hold on;
stem(n, y, 'filled', 'LineWidth', 2);
title('Causality Test Visualization');
xlabel('n'); ylabel('Amplitude');
legend('x[n]', 'y[n] = -x[n]');
grid on;
xticks(-2:2);
%PART (c) LINEARITY TEST
fprintf('\n(c) Testing Linearity...\n');
x1 = 2*n;
x2 = n/3;
a1 = 2;
a2 = 3;
lhs = -(a1*x1 + a2*x2); % System(a1 x1 + a2 x2)
rhs = a1*(-x1) + a2*(-x2); % a1 System(x1) + a2 System(x2)
disp('LHS (system(a1*x1 + a2*x2)) :'); disp(lhs)
disp('RHS (a1*system(x1) + a2*system(x2)) :'); disp(rhs)
if isequal(lhs,rhs)
    fprintf('System is LINEAR.\n');
else
    fprintf('System is NON-LINEAR.\n');
end
figure;
subplot(2,1,1);
stem(n, lhs, 'filled', 'LineWidth', 2);
title('LHS = System(a_1 x_1 + a_2 x_2)');
xlabel('n'); ylabel('Amplitude'); grid on;
xticks(-2:2);
subplot(2,1,2);
stem(n, rhs, 'filled', 'LineWidth', 2);
title('RHS = a_1 System(x_1) + a_2 System(x_2)');
xlabel('n'); ylabel('Amplitude'); grid on;

```

```

xticks(-2:2);
%PART (d) SHIFT INVARIANCE
fprintf('\n(d) Testing Shift Invariance...\n');
n0 = 3;           % shift amount
x_shifted = 2*(n - n0);
y_shifted_input = -x_shifted; % System after shifting input
y_original = -x;
y_shifted_output = y_original; % Conceptual y[n - n0]
disp('Output of shifted input y[n] = -x[n-3]:');
disp([n' y_shifted_input'])
disp('Shifted original output y[n-3]:');
disp([n' y_shifted_output'])
fprintf('System is SHIFT-INVARIANT (shifting input shifts output equally).\n');
figure;
subplot(2,1,1);
stem(n, y_shifted_input, 'filled', 'LineWidth', 2);
title('Output of Shifted Input: y[n] = -x[n - 3]');
xlabel('n'); ylabel('Amplitude'); grid on;
xticks(-2:2);
subplot(2,1,2);
stem(n, y_shifted_output, 'filled', 'LineWidth', 2);
title('Shifted Original Output: y[n - 3]');
xlabel('n'); ylabel('Amplitude'); grid on;
xticks(-2:2);

```

Output:

Input $x[n]$:

-2	-4
-1	-2
0	0
1	2
2	4

Output $y[n] = -x[n]$:

-2	4
-1	2
0	0
1	-2
2	-4

(a) Checking Static or Dynamic...

System is STATIC (memoryless) because $y[n]$ depends only on $x[n]$.

(b) Checking Causality...

System is CAUSAL because output uses only present input $x[n]$.

(c) Testing Linearity...

LHS (system($a_1x_1 + a_2x_2$)) :

10
5
0
-5
-10

RHS ($a_1\text{system}(x_1) + a_2\text{system}(x_2)$) :

10
5
0
-5
-10

System is LINEAR.

(d) Testing Shift Invariance...

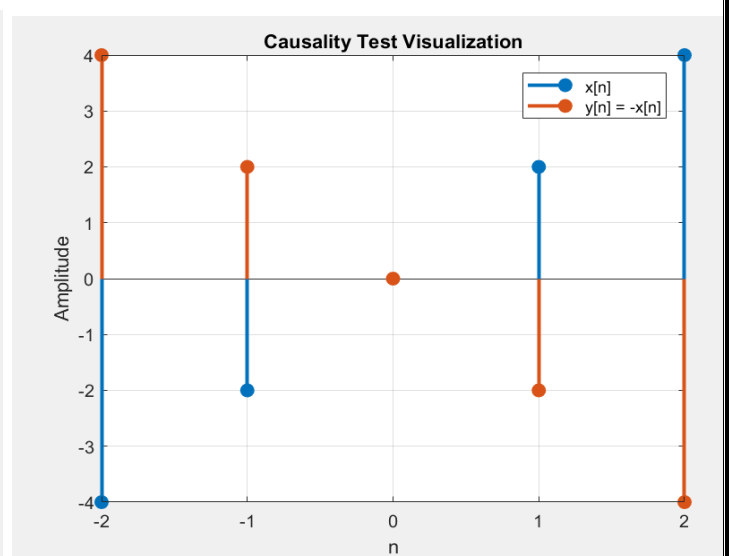
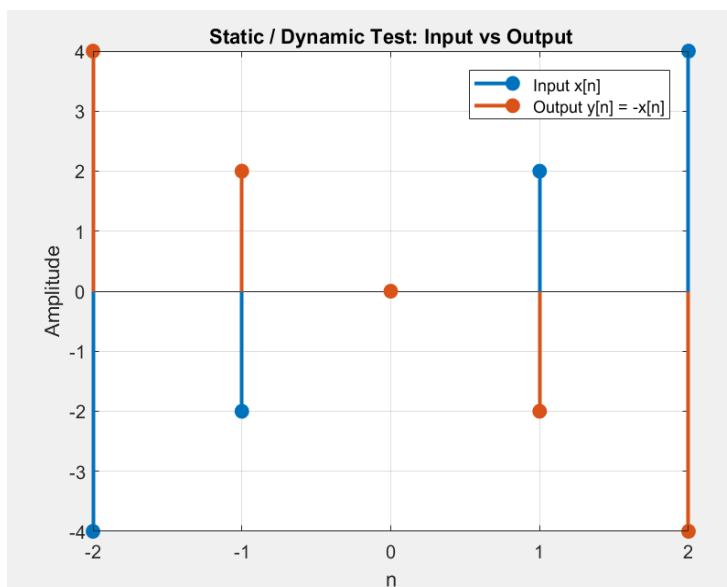
Output of shifted input $y[n] = -x[n-3]$:

-2	10
-1	8
0	6
1	4
2	2

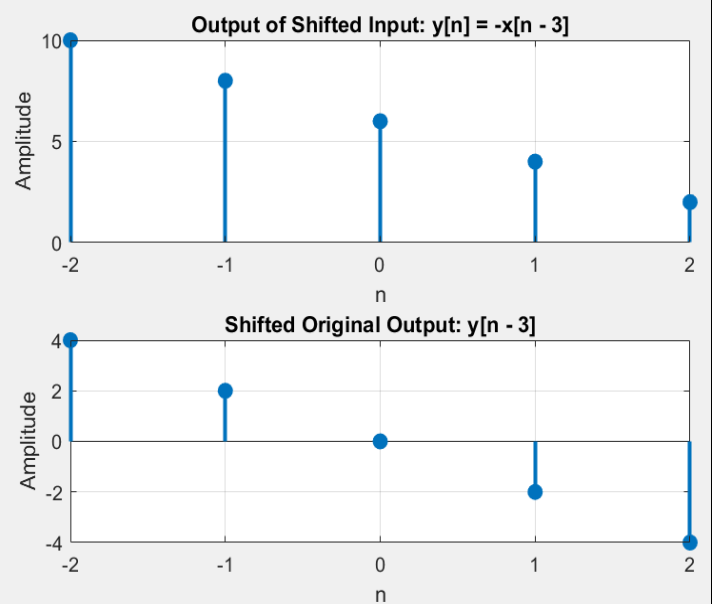
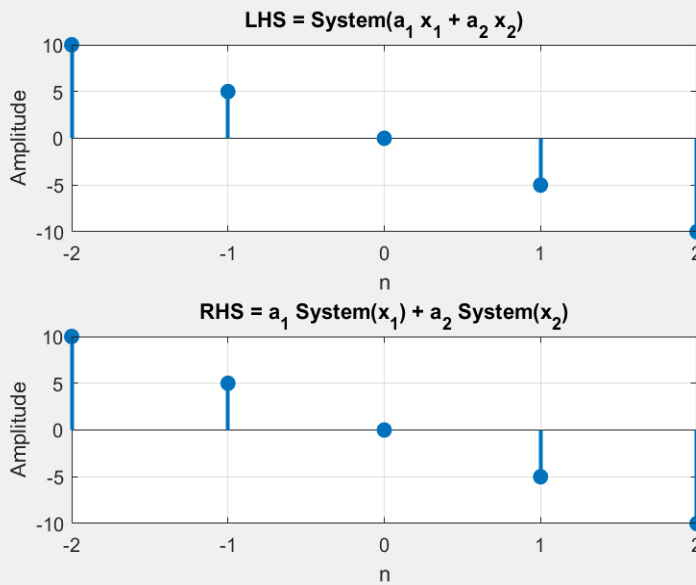
Shifted original output $y[n-3]$:

-2	4
-1	2
0	0
1	-2
2	-4

System is SHIFT-INVARIANT (shifting input shifts output equally).



Linearity Test and Shift Invariance Output:



```
clc; clear; close all;
% Define input signal
n = -2:2;          % Time indices
x = 2*n;           % x[n] = 2*n
y = zeros(size(n)); % Initialize output
% Compute memory system output
for i = 1:length(n)
    if i == 1
        y(i) = x(i); % First sample has no past input
    else
        y(i) = x(i) + x(i-1);
    end
end
% Display signals in console
fprintf('Memory System: y[n] = x[n] + x[n-1]\n\n');
fprintf('n: '); fprintf('%4d ', n); fprintf('\n');
fprintf('x[n]: '); fprintf('%4d ', x); fprintf('\n');
fprintf('y[n]: '); fprintf('%4d ', y); fprintf('\n\n');
% Plot input and output signals together
figure('Name','Memory System Analysis','NumberTitle','off');
```

```

subplot(2,1,1);
stem(n, x, 'r', 'filled', 'LineWidth', 2);
title('Input Signal  $x[n] = 2^n$ ');
xlabel('Time Index n'); ylabel('Amplitude');
grid on; xlim([min(n)-1 max(n)+1]); ylim([min(x)-1 max(x)+1]); xticks(n);
subplot(2,1,2);
stem(n, y, 'b', 'filled', 'LineWidth', 2);
title('Memory System Output  $y[n] = x[n] + x[n-1]$ ');
xlabel('Time Index n'); ylabel('Amplitude');
grid on; xlim([min(n)-1 max(n)+1]); ylim([min(y)-1 max(y)+1]); xticks(n);
% Plot individual sample contributions in separate figures
for i = 1:length(n)
    figure('Name',sprintf('Sample n = %d', n(i)), 'NumberTitle','off');
    hold on;
    % Current input
    stem(n(i), x(i), 'r', 'filled', 'LineWidth', 2);
    % Past input (if exists)
    if i > 1
        stem(n(i-1), x(i-1), 'g', 'filled', 'LineWidth', 2);
    end
    % Output
    stem(n(i), y(i), 'b', 'filled', 'LineWidth', 2);
    % Title and legend
    if i == 1
        title(sprintf('n = %d: y[%d] = x[%d] (no past input)', n(i), n(i), n(i)));
        legend(sprintf('x[%d] (current)', n(i)), ...
            sprintf('y[%d] = %d', n(i), y(i)), 'Location','best');
    else
        title(sprintf('n = %d: y[%d] = x[%d] + x[%d]', n(i), n(i), n(i), n(i-1)));
        legend(sprintf('x[%d] (current)', n(i)), ...
            sprintf('x[%d] (past)', n(i-1)), ...
            sprintf('y[%d] = %d', n(i), y(i)), 'Location','best');
    end
    grid on;
    xlim([min(n)-1 max(n)+1]); ylim([min([x y])-2 max([x y])+2]);
    xlabel('Time Index n'); ylabel('Amplitude');
    xticks(n);
end

```

```
% Detailed calculation table
fprintf('Detailed Calculations:\n');
fprintf('n  x[n]  x[n-1]  y[n] = x[n]+x[n-1]  System Type\n');
fprintf('-----\n');
for i = 1:length(n)
    if i == 1
        fprintf('%d  %2d  --  %2d (no past)  Memory System\n', n(i), x(i), y(i));
    else
        fprintf('%d  %2d  %2d  %2d + %2d = %2d  Memory System\n', ...
            n(i), x(i), x(i-1), x(i), x(i-1), y(i));
    end
end
end
```

Command Window

Memory System: $y[n] = x[n] + x[n-1]$

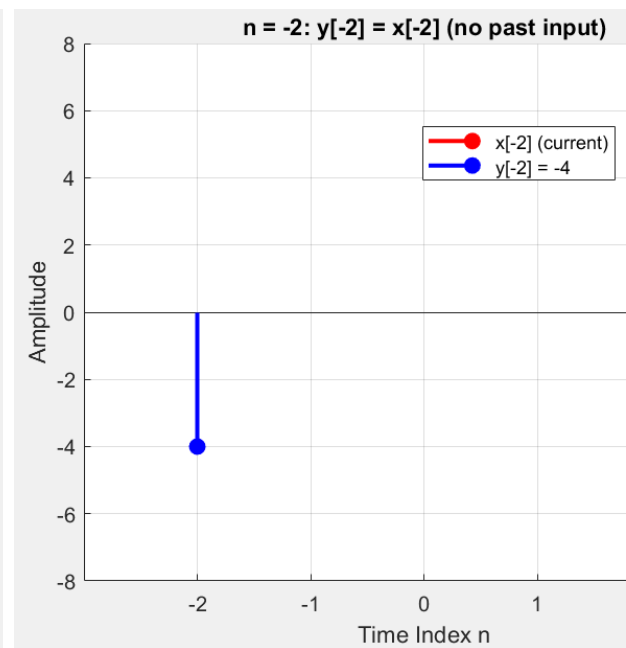
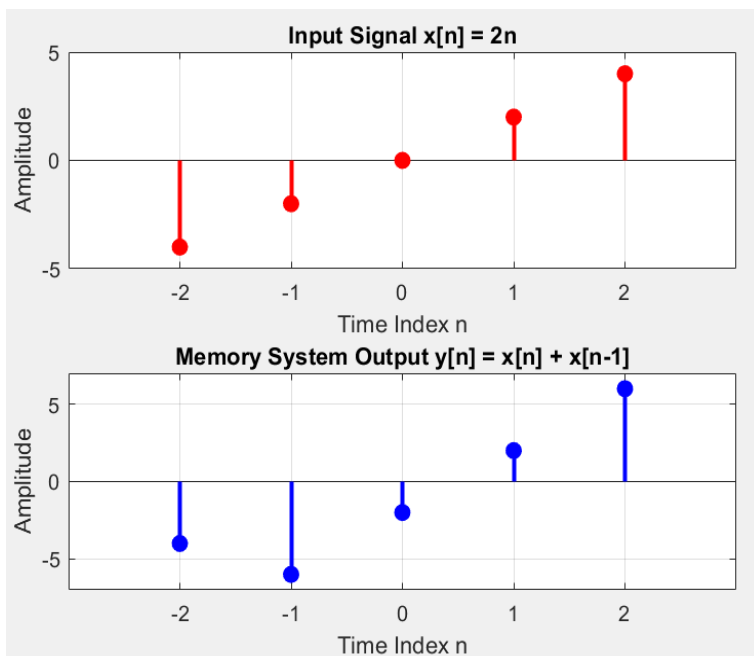
```
n:      -2  -1   0   1   2
x[n]:   -4  -2   0   2   4
y[n]:   -4  -6  -2   2   6
```

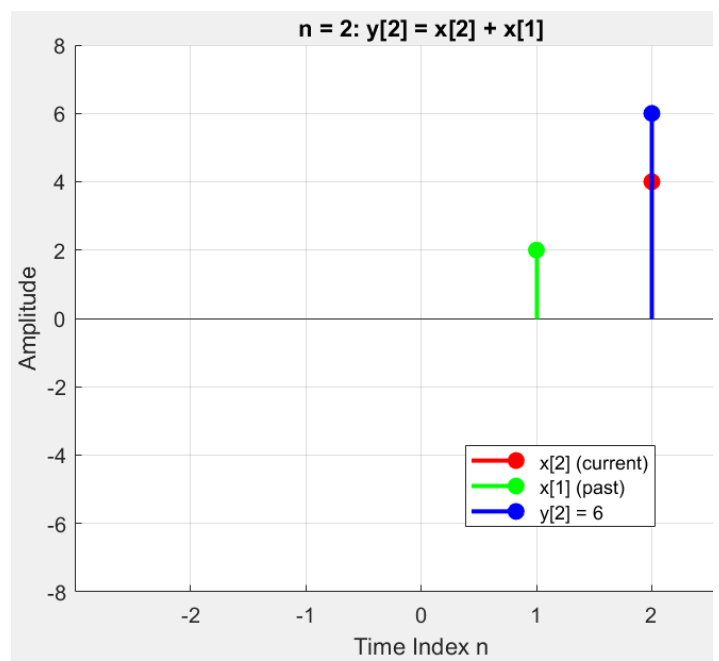
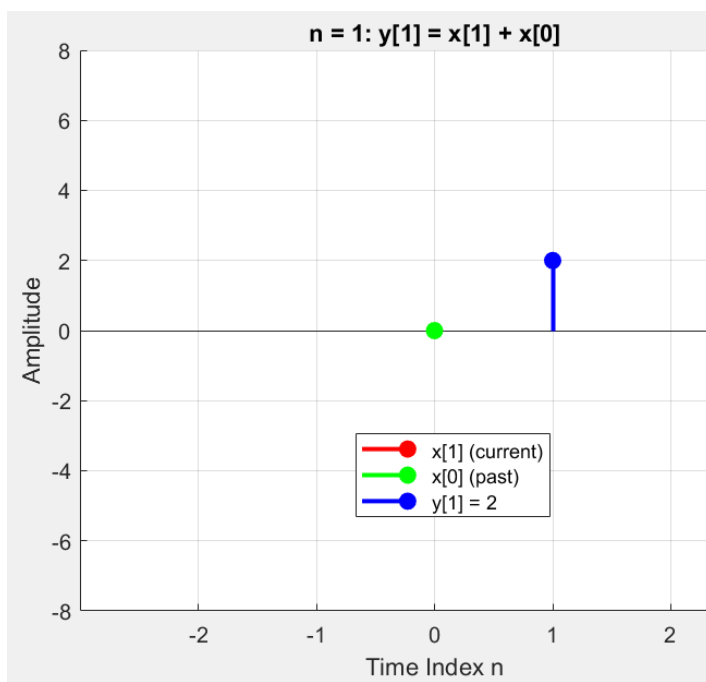
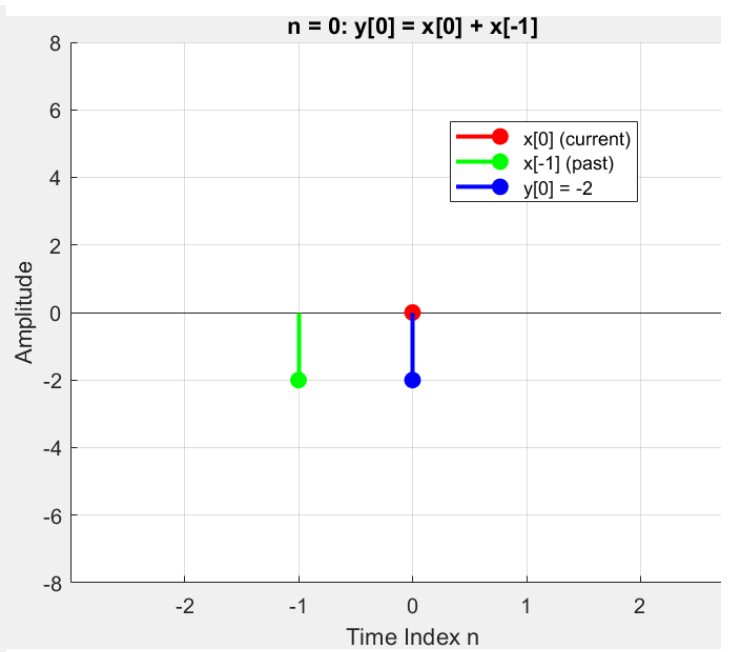
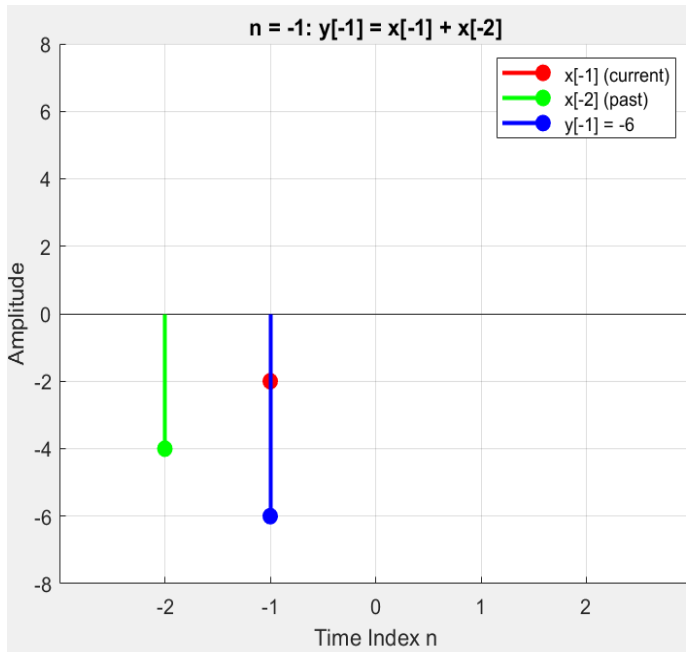
Detailed Calculations:

n	x[n]	x[n-1]	y[n] = x[n]+x[n-1]	System Type
-2	-4	--	-4 (no past)	Memory System
-1	-2	-4	-2 + -4 = -6	Memory System
0	0	-2	0 + -2 = -2	Memory System
1	2	0	2 + 0 = 2	Memory System
2	4	2	4 + 2 = 6	Memory System

fx >>

Output Graph:





Non-Causal System Example:

Code:

```
clc; clear; close all;
% INPUT SIGNAL
n = -3:3;
x = 2*n;    % x[n] = 2n
% NON-CAUSAL SYSTEM: y[n] = x[n+1] + x[n] (depends on FUTURE input)
% For demonstration, let's use: y[n] = x[n+1] (depends on next sample)
y = zeros(size(n));
% Calculate output for non-causal system
for i = 1:length(n)
    if i < length(n)
        y(i) = x(i+1); % y[n] = x[n+1] - depends on FUTURE input
    else
        y(i) = 0;    % For last sample, no future value exists
    end
end
% Display the signals
fprintf('Non-Causal System: y[n] = x[n+1]\n\n');
fprintf('n: ');
fprintf('%4d ', n);
fprintf('\nx[n]: ');
fprintf('%4d ', x);
fprintf('\ny[n]: ');
fprintf('%4d ', y);
fprintf('\n\n');

% Plot all samples together to show the time relationship
figure;
% Plot input signal
subplot(2,1,1);
stem(n, x, 'filled', 'r', 'LineWidth', 2);
```

```

title('Input Signal: x[n] = 2n');
xlabel('Time Index n');
ylabel('Amplitude');
grid on;
xlim([min(n)-0.5 max(n)+0.5]);
ylim([min(x)-1 max(x)+1]);
% Plot output of non-causal system
subplot(2,1,2);
stem(n, y, 'filled', 'b', 'LineWidth', 2);
title('Non-Causal System Output: y[n] = x[n+1]');
xlabel('Time Index n');
ylabel('Amplitude');
grid on;
xlim([min(n)-0.5 max(n)+0.5]);
ylim([min(y)-1 max(y)+1]);
% Plot individual samples to demonstrate non-causality
for i = 1:length(n)-1 % Skip last sample as it has no future
    figure;
    % Plot current input and corresponding future input
    stem(n(i), x(i), 'filled', 'r', 'LineWidth', 2); hold on;
    stem(n(i+1), x(i+1), 'filled', 'g', 'LineWidth', 2);
    stem(n(i), y(i), 'filled', 'b', 'LineWidth', 2);
    title(['Non-Causal System at n = ', num2str(n(i)), ': y[', num2str(n(i)), '] = x[', num2str(n(i+1)), ']']);
    xlabel('Time Index n');
    ylabel('Amplitude');
    legend(['x[', num2str(n(i)), '] (current)', 'x[', num2str(n(i+1)), '] (future)', ...
        'y[', num2str(n(i)), '] = x[', num2str(n(i+1)), ']', 'Location', 'best');
    grid on;
    xlim([min(n)-0.5 max(n)+0.5]);
    ylim([min([x y])-1 max([x y])+1]);
end

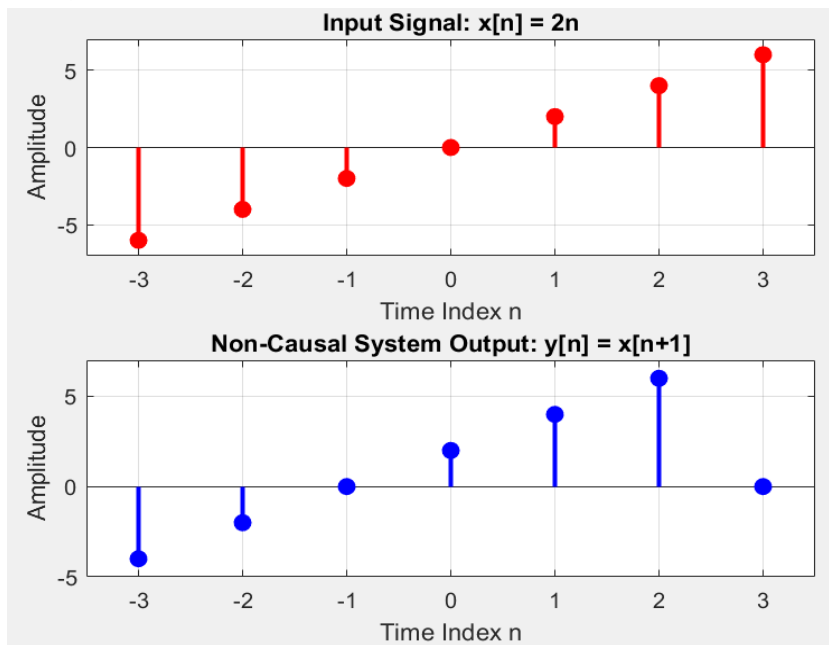
```

Command Window

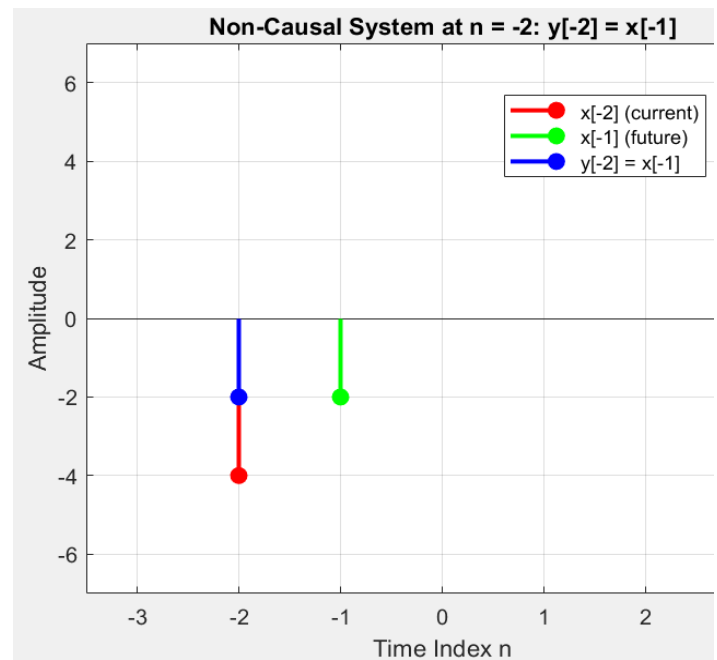
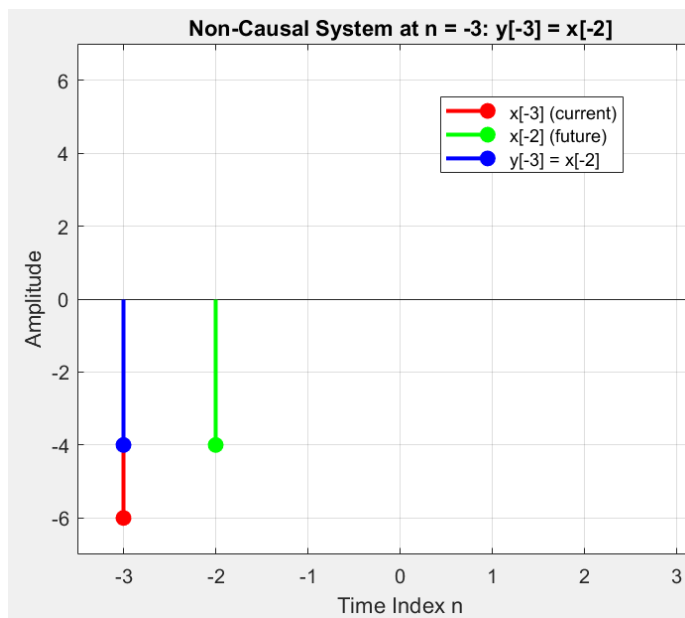
Non-Causal System: $y[n] = x[n+1]$

n:	-3	-2	-1	0	1	2	3
x[n]:	-6	-4	-2	0	2	4	6
y[n]:	-4	-2	0	2	4	6	0

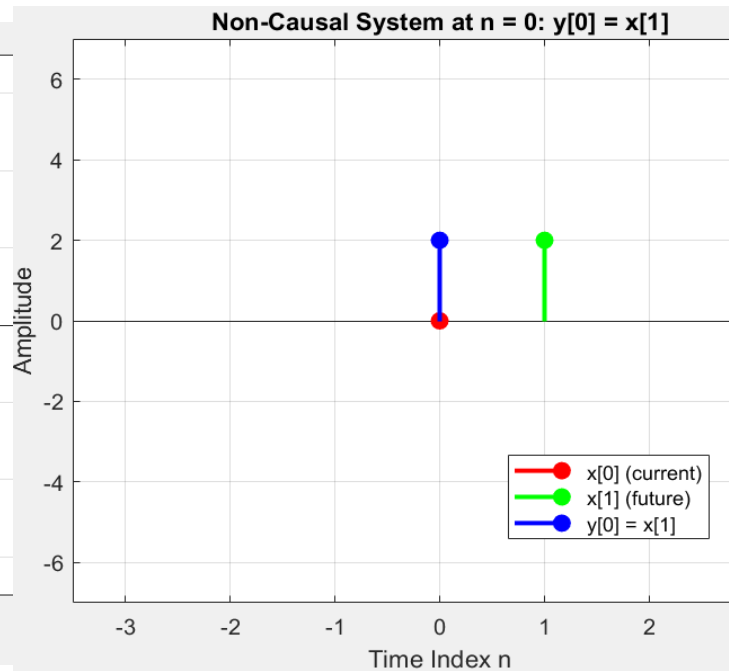
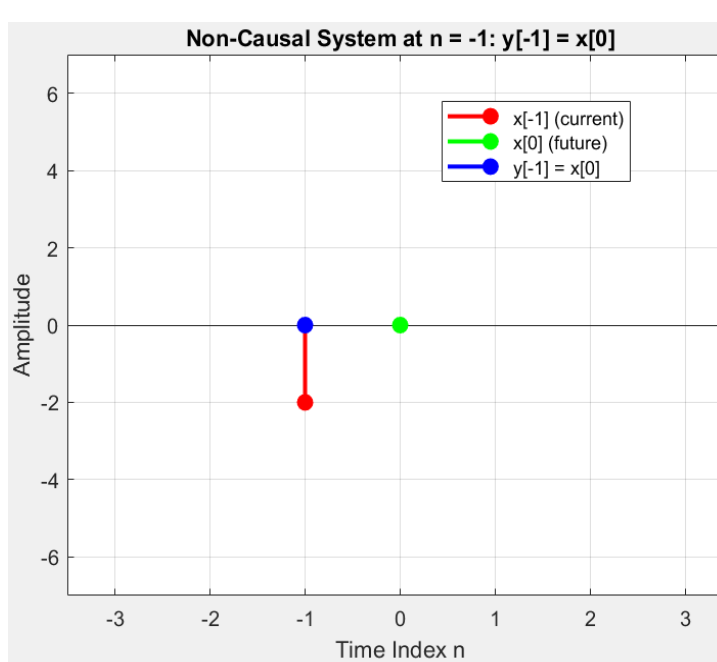
Output:



Non Causal System At $n = -3, -2$:



Non Causal System At $n = -1, 0$:



Non Causal System At $n = 1, 2$:

